Exercise 2 - To Embed or Not to Embed ...

Building Word Embeddings with PyTorch

Deadlines

Deadline for Exercise 2: **Sunday, 26.10.2025, 23:59 (Zurich Time)**.

Deadline for the peer review: **Monday, 03.11.2025, 23:59 (Zurich Time).**

- Note, the peer reviews will be assigned one day after the exercise deadline. You will find instructions for the peer review process and declaration of generative AI usage at the end of this document.

Learning goals

This exercise introduces you to PyTorch and how we can use it to create our own corpus-specific word embeddings. By completing this exercise you should …

- … understand the basic building blocks for training models in PyTorch.
- … understand what different word embeddings are and how you train them.
- … think about how one can evaluate word embeddings.

Please keep in mind that you can always consult and use the exercise forum if you get stuck (note that we have a separate forum for the exercises).

Deliverables

Your solutions for each part should be submitted as self-contained iPython notebooks (ipynb files). The notebooks should contain your well-documented, EXECUTED, and EXECUTABLE code. That way, your reviewers can view and execute your code without potential dependency issues and/or installing new packages or versions of packages.

Please hand in your code as a notebook with a report section at the end of the notebook as markdown. Name your notebook as

- ex02_wordembeddings.ipynb

The .ipynb file should contain your well documented AND EXECUTABLE code.

For this exercise, we recommend working with Google Colab or Kaggle, where you have access to GPUs to train the embedding models. On a T4-GPU, it may take up to an hour for training hotel reviews dataset and up to 3 hours for training Sci-Fi dataset. You can try to solve the exercise on your own computer fully. However, be aware that training without a GPU may take a long time.

NOTE: ONLY SWITCH from CPU to a GPU instance **AFTER** you have confirmed that your training procedure is working correctly. This may require rerunning the setup and preprocessing again (if you don't store intermediate processing files), but this is a smaller issue than running out of GPU hours. Before that, either develop your code locally on your own computer or use Google Colab or Kaggle in a non-GPU runtime.

Please ensure you run your final code on Kaggle on a GPU runtime. Make the GPU selection from “Settings” → “Accelerator” → “GPU T4 x2” and then use the GPU hardware accelerator.

Colab and Kaggle will disconnect you from the runtime due to inactivity after some time, to prevent this please follow the instructions here which work for both Colab and Kaggle.

We assume that the data files are in the same folder as the scripts, e.g.

- ex02_wordembeddings.ipynb
- scifi.txt
- tripadvisor_hotel_reviews.csv

You will also have to write a short lab report at the end of each script. The lab report should contain a detailed description of the approaches you have used to solve this exercise. Please also include results. In this exercise description, we highlight places in green where we expect a statement about an issue in your lab report.

Please note:

- Your peers need to be able to run your code. If it does not work, you will not be able to obtain the maximum number of points.
- The cells must have already been run and the output must be visible to anyone checking your notebook without having to run the code again.
- DO NOT submit the data files!
- Please make sure that your answers to the specific questions outlined in the notebook and denoted with “📝❓” are clearly marked. If the assessors cannot easily find the answers they need, you can’t get the maximum score. We recommend including these clearly in your lab report.

Data

You will work with a reduced version of “Trip Advisor hotel reviews” and “Sci-fi stories”. The files are hosted under the following links:

- tripadvisor_hotel_reviews_reduced.csv
- scifi_reduced.txt

The above URL can be used to load the data directly into your notebook - as you already did in Exercise 1.

Part 1 - Train your CBOW embeddings for both datasets

Go to this Colaboratory Python Notebook, create a copy of it and complete the missing code in the exercise section – which means implementing a CBOW model in PyTorch. Then, change the code so that it takes the Trip advisor hotel reviews text as input and produces word embeddings for the hotel reviews.

Make sure that the code is understandable by either using comments for classes and methods or by explaining the code in text cells of the notebook.

It is up to you to decide on the specific preprocessing steps (splitting sentences, tokenizing words, dealing with punctuation, character casing, removing stopwords (perhaps!?), etc.).

1. Describe your decisions (preprocessing, class structure) in the lab report.

You will train three models:

- CBOW2 with a context width of 2 (in both directions) for the **Hotel Reviews dataset.**
- CBOW5 with a context width of 5 (in both directions) for the **Hotel Reviews dataset.**
- and CBOW2 with a context width of 2 (in both directions) for the **Sci-Fi story dataset.**

In order to obtain useful embeddings without training too long, we recommend an embedding size of 50 and 12-15 epochs on the hotel reviews dataset. On the Sci-fi dataset 3 epochs are enough.

Train CBOW5 on the **Hotel Reviews dataset** with a context width of 5 (in both directions).

2. Are predictions made by the model sensitive towards the context size?

(**optional read**) - This paper (here) provides an insight on how to choose a minimum embedding size while still obtaining useful representations.

Part 2 - Test your embeddings

Word embeddings are not easy to evaluate automatically without suitable test sets. For this exercise, we inspect them manually to get a feel for whether they capture what we think they should capture. Computing the nearest neighbours (see below) for a word allows us to get an intuition of the semantic vector space. Do the following for CBOW2 and, optionally, for CBOW5:

3. For the hotel reviews dataset, choose 3 nouns, 3 verbs, and 3 adjectives. Make sure that some nouns/verbs/adjectives occur frequently in the corpus and that others are rare. For each of the 9 chosen words, retrieve the 5 closest words according to your trained CBOW2 model. List them in your report and comment on the performance of your model: do the neighbours the model provides make sense? Discuss.
4. Repeat what you did in 2. for the Sci-fi dataset.
5. How does the quality of the hotel review-based embeddings compare with the Sci-fi-based embeddings? Elaborate.
6. Choose 2 words and retrieve their 5 closest neighbours according to hotel review-based embeddings and the Sci-fi-based embeddings. Do they have different neighbours? If yes, can you reason why?
7. What are the differences between CBOW2 and CBOW5 ? Can you "describe" them?

Function for nearest neighbour computation

```
import torch.nn as nn

def get_closest_word(word, topn=5):
    word_distance = []
    emb = net.embeddings_target
    pdist = nn.PairwiseDistance()
    i = word_to_index[word]
    lookup_tensor_i = torch.tensor([i], dtype=torch.long)
    v_i = emb(lookup_tensor_i)
    for j in range(len(vocabulary)):
        if j != i:
            lookup_tensor_j = torch.tensor([j], dtype=torch.long)
            v_j = emb(lookup_tensor_j)
            word_distance.append((index_to_word[j], float(pdist(v_i, v_j))))
    word_distance.sort(key=lambda x: x[1])
    return word_distance[:topn]
```

“net” above corresponds to the CBOW class in the Colab notebook. Note that you might have to adapt the code above, so it fits your initialization of the model.

Part 3 - Comparison global embeddings

In this final part, you will use pretrained GloVe embeddings to explore how large-scale, general-domain word representations differ from the domain-specific CBOW embeddings you trained in Parts 1 and 2.

Background
GloVe (Global Vectors for Word Representation) (Pennington et al. 2014) is a count-based embedding model trained on very large text corpora such as Wikipedia, Common Crawl, and Twitter.
Unlike your CBOW model, which learns to predict a target word from its local context window, GloVe learns embeddings by modeling global co-occurrence statistics (how frequently words appear together) across the entire corpus.

Same as with CBOW, each word in GloVe is represented by one static vector, meaning that its embedding does not change depending on sentence context (e.g., "bank" will have the same vector in river bank and financial bank).

Note: Loading pretrained GloVe embeddings does not require a GPU.
This step runs efficiently on the CPU.
Before running this section, switch your Colab or Kaggle runtime back to CPU to conserve GPU compute units. Make sure to not lose any progress from previous steps when doing so.

Task

1. Load a pretrained GloVe model: Use the provided code snippet to download and load the 100-dimensional GloVe vectors (glove.6B.100d.txt). The script converts the GloVe format into the Word2Vec format used by Gensim and then loads it as a KeyedVectors object.
2. Select words to analyze: Reuse the same nouns, verbs, and adjectives you investigated in Part 2 (for example, from the hotel reviews dataset).
3. Retrieve nearest neighbours: For each of your chosen words, the provided code snippet uses the GloVe model's .most_similar() method to list the five most similar words according to cosine similarity.
4. Interpret and compare:
    A. Compare the GloVe neighbours with those produced by your CBOW2 (and optionally CBOW5) models.
    B. Do the GloVe neighbours seem more general or semantically broader?
    C. Are the CBOW neighbours more domain-specific to hotel reviews or the Sci-Fi text?
    D. Discuss how training data size and training objective (predictive vs. count-based) might explain the observed differences.

Submission & Peer Review Guidelines:

For all exercises, peer reviews will be carried out on OLAT.

As soon as the deadline for handing in the exercise expires, you will have time to review the submissions of your peers. You need to do **1 review per person** to get the maximum points for this exercise.

Here are some additional rules:

- **All file submissions are anonymous (for peer review purposes): Do not write your name into your Jupyter Notebooks.**
- **EVERY** member of each team submits on OLAT.
- Please submit a zip folder containing all the deliverables.

Groups & Peer Reviews:

- You should work in the same group for all exercises. Each member should contribute equally!
- If you do not submit your peer review, you will receive a -0.25 point penalty on the final grade of the exercise.
- Please use full sentences when giving feedback. Please write the peer review in English.
- Provide **constructive** and helpful comments to your peers! If you criticise something, you should also be able to provide actionable and realistic suggestions for how something can be improved.
- Please answer all the pre-designed review questions of the peer review to the best of your ability.

Declaration of Usage of Generative AI

For every exercise, please append a short **"Declaration of Usage of Generative AI"** section at the end of your submission. In this declaration, briefly (a few sentences are sufficient!) describe **how** and **to what extent** you used Generative AI tools (e.g., ChatGPT, Copilot, Gemini, etc.) during the completion of this exercise.

Your statement should address the following points:

- **Source code and documentation:** How was Generative AI used to assist in developing, debugging, or documenting your code?
- **Written report:** Was Generative AI used to draft, revise, or post-edit the written components of your submission?
- **Learning support:** Did you use Generative AI as an assistant for concept clarification, idea exploration, or learning guidance?